

TP 5 - Équations différentielles - Dynamique des populations

*Modèle prédateur-proie
Lotka-Volterra*

On note $N(t)$ le nombre de proies et $P(t)$ le nombre de prédateurs. Le modèle de Lotka-Volterra est régi par le système d'équations différentielles suivant.

$$\begin{cases} N'(t) &= (a - b P(t)) N(t), \\ P'(t) &= (-p + q N(t)) P(t), \end{cases}$$

où les coefficients a, b, p, q sont des constantes positives. Nous prendrons $a = 3, b = 2, p = 2.5, q = 2$. Nous allons résoudre ce système par trois méthodes numériques.

1- Méthode d'Euler explicite

Nous considérons le problème $y'(t) = f(t, y(t))$ avec $y(0) = y_0 = [N_0, P_0]$ et $t \in [t_0, t_f]$. La méthode d'Euler est alors définie par la relation de récurrence :

$$y_{k+1} = y_k + h f(t_k, y_k), \quad \text{pour } k = 0, n,$$

avec $h = (t_f - t_0)/n$, $t_k = t_0 + kh$ et y_k une approximation de $y(t_k)$. En Matlab cela s'écrit simplement de la manière suivante :

```
function [tt,y]=meuler(t0,tf,n,y0,f)
% Commentaires
h=abs(tf-t0)/n ;
y(1, :)=y0+h*f(t0,y0)' ;
tt(1)=t0+h ;
for k=1 :n-1
    y(k+1, :)=y(k, :)+h*f(tt(k),y(k, :))' ;
    tt(k+1)=tt(k)+h ;
end
tt=[t0,tt] ;y=[y0 ;y] ;
```

La variable `f` contient le pointeur sur la fonction $f(t, y)$ (qui ici ne dépend pas de t) définissant le système (dans le fichier `flv.m`) :

```
function np = flv(t,x)
%
global a b p q ;
np=[(a-b*x(2))*x(1) ; (-p+q*x(1))*x(2)] ;
```

L'instruction `global a b p q` ; est aussi à mettre dans le script appelant où sont définis ces paramètres. L'appel de la fonction `meuler` sera `meuler(t0,tf,n,y0,@flv)` .

Nous prenons $t_0 = 0, t_f = 16$ et comme condition initiale $N_0 = 1.5$ et $P_0 = 3.5$. Résoudre le système par la méthode d'Euler et représenter P en fonction de N pour

$n = 10000, 2000, 500, 350, 300$. Puis pour $n = 2000$ fixé, tracer sur une même figure le portrait de phase pour $P_0 = 1.5, 2.5, 3.5$ en signalant par un rond rouge le point correspondant à t_0 et un rond vert celui correspondant à t_f .

2- Méthode d'Euler implicite

La méthode d'Euler implicite est définie par la relation de récurrence :

$$y_{k+1} = y_k + hf(t_{k+1}, y_{k+1}), \quad \text{pour } k = 0, n-1,$$

avec $h = (t_f - t_0)/n$, $t_k = t_0 + kh$ et y_k une approximation de $y(t_k)$. C'est-à-dire qu'à chaque étape il y a un point fixe à calculer. Le plus simple à programmer est la récurrence $u_{j+1} = \Phi(u_j)$ pour une application contractante. En Matlab cela s'écrit simplement de la manière suivante :

```
function [tt,y]=eulimp(t0,tf,n,y0,f,tol,nitmax)
% Commentaires décrivant la fonction
h=abs(tf-t0)/n ;
tt(1)=t0+h ; zn=y0 ; znp1=y0+h*f(tt(1),zn)' ; nit=1 ;
while norm(znp1-zn,inf)>tol ;
    zn=znp1 ; znp1=y0+h*f(tt(1),zn)' ; nit=nit+1 ;
    if (nit > nitmax)
        disp('Nombre maximal d iterations atteint') ; return ;
    end
end
y(1, :)=znp1 ;
for i=1 :n-1
    tt(i+1)=tt(i)+h ;
    zn=y(i, :) ; znp1=y(i, :)+h*f(tt(i+1),zn)' ; nit=1 ;
    while norm(znp1-zn,inf)>tol ;
        zn=znp1 ; znp1=y(i, :)+h*f(tt(i+1),zn)' ; nit=nit+1 ;
        if (nit > nitmax)
            disp('Nombre maximal d iterations atteint') ; return ;
        end
    end
    y(i+1, :)=znp1 ;
end
tt=[t0,tt] ; y=[y0 ; y] ;
```

où tol est la précision demandée pour le point fixe et $nitmax$ son nombre maximal d'itérations (on pourra prendre $tol = 1e-10$ et $nitmax = 100$).

Pour $t_0 = 0$, $t_f = 16$, $N_0 = 1.5$, $n = 2000$, tracer sur une même figure le portrait de phase pour $P_0 = 1.5, 2.5, 3.5$ en signalant par un rond rouge le point correspondant à t_0 et un rond vert celui correspondant à t_f .

3- Méthode de Runge-Kutta (2,3) explicite : ode23

Avec les mêmes données que précédemment, résoudre le système puis tracer le portrait de phase pour $P_0 = 1.5, 2.5, 3.5$. Faire de même avec $t_f = 100$. Représenter la variation des effectifs en fonction du temps ($N(t), P(t)$) pour $t_f = 16$ et $P_0 = 3.5$.

Résultats - TP 5

